Java Sessional: Experiments 1–8

Quick revision PDF • Code + file names + key points

Before writing: Save each program in a file matching its public class name. Compile with `javac FileName.java`, then run with `java ClassName`. Experiment 1 needs the image file in the correct location.

1. Display Image using JFrame

File name: DisplayImage.java

```
import javax.swing.*;

public class DisplayImage {
    public static void main(String[] args) {
        JFrame f = new JFrame("Display Image");
        ImageIcon img = new ImageIcon("pic.jpg");
        JLabel label = new JLabel(img);
        f.add(label);
        f.setSize(500, 400);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Remember: JFrame creates the window; ImageIcon loads the image; JLabel displays it. Keep pic.jpg in the same folder as the program.

2. Restaurant Bill

File name: RestaurantBill.java

```
import javax.swing.*;
import java.awt.event.*;

public class RestaurantBill {
    public static void main(String[] args) {
        JFrame f = new JFrame("Restaurant Bill");
        JLabel l1 = new JLabel("Food Name:");
        JLabel l2 = new JLabel("Price:");
        JLabel l3 = new JLabel("Quantity:");
        JTextField t1 = new JTextField();
        JTextField t2 = new JTextField();
        JTextField t3 = new JTextField();
        JButton b = new JButton("Calculate Bill");

        l1.setBounds(30, 30, 100, 30);
        t1.setBounds(130, 30, 150, 30);
        l2.setBounds(30, 70, 100, 30);
        t2.setBounds(130, 70, 150, 30);
        l3.setBounds(30, 110, 100, 30);
        t3.setBounds(130, 110, 150, 30);
        b.setBounds(90, 160, 150, 30);

        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    String food = t1.getText();
                    double price = Double.parseDouble(t2.getText());
                    int qty = Integer.parseInt(t3.getText());
                    double total = price * qty;
                    JOptionPane.showMessageDialog(f,
                        "Food: " + food +
                        "\\nTotal Bill: " + total + " Taka");
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(f,
                        "Enter valid price and quantity!");
                }
            }
        });

        f.add(l1); f.add(t1); f.add(l2); f.add(t2);
        f.add(l3); f.add(t3); f.add(b);
        f.setSize(350, 260);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Remember: Bill = price × quantity. setBounds(x, y, width, height) sets position and size; setLayout(null) enables manual positioning.

3. Student Form in GUI

File name: StudentForm.java

```
import javax.swing.*;
import java.awt.event.*;

public class StudentForm {
    public static void main(String[] args) {
        JFrame f = new JFrame("Student Form");
        JLabel l1 = new JLabel("Name:");
        JLabel l2 = new JLabel("ID:");
        JLabel l3 = new JLabel("Department:");
        JTextField t1 = new JTextField();
        JTextField t2 = new JTextField();
        JTextField t3 = new JTextField();
        JButton b = new JButton("Submit");

        l1.setBounds(30, 30, 100, 30);
        t1.setBounds(130, 30, 150, 30);
        l2.setBounds(30, 70, 100, 30);
        t2.setBounds(130, 70, 150, 30);
        l3.setBounds(30, 110, 100, 30);
        t3.setBounds(130, 110, 150, 30);
        b.setBounds(90, 160, 100, 30);

        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(f,
                    "Name: " + t1.getText() +
                    "\\nID: " + t2.getText() +
                    "\\nDepartment: " + t3.getText());
            }
        });

        f.add(l1); f.add(t1); f.add(l2); f.add(t2);
        f.add(l3); f.add(t3); f.add(b);
        f.setSize(350, 250);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Remember: JTextField takes input; JButton triggers the ActionListener; JOptionPane shows the entered student information.

4. Simple Calculator in GUI

File name: SimpleCalculator.java

```
import javax.swing.*;
import java.awt.event.*;

public class SimpleCalculator {
    public static void main(String[] args) {
        JFrame f = new JFrame("Simple Calculator");
        JTextField t1 = new JTextField();
        JTextField t2 = new JTextField();
        JButton add = new JButton("+");
        JButton sub = new JButton("-");
        JButton mul = new JButton("*");
        JButton div = new JButton("/");

        t1.setBounds(30, 30, 100, 30);
        t2.setBounds(150, 30, 100, 30);
        add.setBounds(30, 80, 50, 30);
        sub.setBounds(90, 80, 50, 30);
        mul.setBounds(150, 80, 50, 30);
        div.setBounds(210, 80, 50, 30);

        ActionListener calc = new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    double a = Double.parseDouble(t1.getText());
                    double b = Double.parseDouble(t2.getText());
                    String op = e.getActionCommand();
                    double result = 0;
                    if (op.equals("+")) result = a + b;
                    else if (op.equals("-")) result = a - b;
                    else if (op.equals("*")) result = a * b;
                    else if (op.equals("/")) {
                        if (b == 0) {
                            JOptionPane.showMessageDialog(f, "Cannot divide by zero");
                            return;
                        }
                        result = a / b;
                    }
```

```
                    JOptionPane.showMessageDialog(f, "Result: " + result);
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(f, "Enter valid numbers");
                }
            }
        };
        add.addActionListener(calc);
        sub.addActionListener(calc);
        mul.addActionListener(calc);
        div.addActionListener(calc);

        f.add(t1); f.add(t2); f.add(add); f.add(sub); f.add(mul); f.add(div);
        f.setSize(300, 170);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Remember: The calculator reads two numbers, checks which button was clicked, performs +, -, * or /, and displays the result.

5. Thread suspend/resume/stop illustration

File name: ThreadControlDemo.java

```
class MyThread extends Thread {
    private boolean paused = false;
    private boolean stopped = false;

    synchronized void pauseThread() {
        paused = true;
    }
    synchronized void resumeThread() {
        paused = false;
        notify();
    }
    synchronized void stopThread() {
        stopped = true;
        paused = false;
        notify();
    }

    public void run() {
        int i = 1;
        while (true) {
            synchronized (this) {
                if (stopped) break;
                while (paused && !stopped) {
                    try { wait(); } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
                if (stopped) break;
            }
            System.out.println("Count: " + i++);
            try { Thread.sleep(500); } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        System.out.println("Thread stopped");
    }
}

public class ThreadControlDemo {
    public static void main(String[] args) throws Exception {
        MyThread t = new MyThread();
        t.start();
        Thread.sleep(1500);
        t.pauseThread();
        System.out.println("Paused");
        Thread.sleep(1500);
        t.resumeThread();
        System.out.println("Resumed");
        Thread.sleep(1500);
        t.stopThread();
    }
}
```

Remember: This safe demonstration imitates suspend/resume/stop using pauseThread(), resumeThread() and stopThread(). Thread.suspend(), resume() and stop() are deprecated/unsafe.

6. Create a Thread Class

File name: ThreadDemo.java

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Remember: Extend Thread and override run(). Calling start() begins a new thread; the JVM then executes run().

7. Illustrate yield(), sleep() and stop()

File name: ThreadMethods.java

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println(i);
            Thread.yield();
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                return;
            }
        }
    }
}

public class ThreadMethods {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
        try {
            Thread.sleep(2000);
            t.stop(); // deprecated/unsafe; only for syllabus demonstration
            System.out.println("Stopped");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Remember: yield() gives other runnable threads a chance; sleep(500) pauses the current thread for about 500 ms; stop() forcibly terminates a thread and is deprecated/unsafe.

8. Use Priority of Thread

File name: ThreadPriority.java

```
class ThreadPriority extends Thread {
    ThreadPriority(String name, int priority) {
        setName(name);
        setPriority(priority);
    }

    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + ": " + i);
        }
    }

    public static void main(String[] args) {
        ThreadPriority t1 = new ThreadPriority("Low", 1);
        ThreadPriority t2 = new ThreadPriority("Normal", 5);
        ThreadPriority t3 = new ThreadPriority("High", 10);
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Remember: Java thread priority ranges from 1 (MIN_PRIORITY) to 10 (MAX_PRIORITY); 5 is NORM_PRIORITY. Higher priority does not guarantee that a thread runs first.

Fast Viva Revision

JFrame: Creates a window.

JLabel: Displays text or an image.

JTextField: Takes text input.

JButton: Creates a clickable button.

ActionListener: Handles button-click events.

setBounds(x, y, width, height): Sets component position and size.

setLayout(null): Allows manual positioning with setBounds().

Thread: A lightweight unit of execution.

start(): Starts a new thread; then run() executes.

run(): Contains the thread's task.

sleep(): Pauses the current thread for a specified time.

yield(): Gives other runnable threads a chance to run.

Priority: A scheduling hint; it does not guarantee execution order.

Thread creation: Common methods: extend Thread or implement Runnable.

Note: Experiment 5 uses safe pause/resume/stop methods to demonstrate the concepts. If your instructor requires a specific code from the lab sheet, follow that exact lab-sheet version.